

Sprint 1A — Voice Ingestion Pipeline

A file-by-file, function-by-function walkthrough of the setup, VAD, and windowing work for the SIH voice-deepfake-detection project.

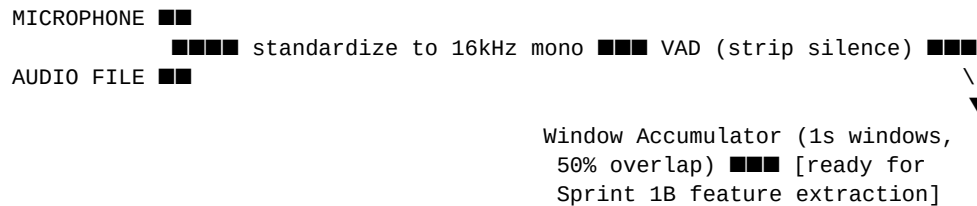
Scope covered: project setup, dependencies & Docker, central config, microphone capture, multi-format file loading, WebRTC voice activity detection, and the overlapping-window accumulator.

Not covered here (belongs to other sprints): feature extraction, dataset download/preprocessing (Sprint 1B), model training/evaluation (Sprint 2), and the WebSocket streaming server / risk aggregation (Sprint 3).

1. The Big Picture

Sprint 1A is the **entry point** of the whole detection system. Before any AI model can decide 'real voice' vs 'AI voice', raw audio has to be captured, cleaned up, and cut into consistent, predictable chunks. That is everything in this document.

The flow, end to end:



Why each stage exists

- **Standardize format** — different files come in at different sample rates, channel counts, and formats (WAV/MP3/M4A/OGG/FLAC). Everything downstream assumes one consistent format: 16kHz, mono. Without this step, every other module would need format-handling logic duplicated everywhere.
- **VAD (silence removal)** — a real phone call is full of pauses, breathing, background noise. Feeding all of that into a detection model wastes compute and adds noise to the decision. VAD keeps only the parts where someone is actually speaking.
- **Windowing** — ML models need fixed-size input. A 1-second window is a good balance: long enough to capture meaningful voice characteristics, short enough for real-time / low-latency response. 50% overlap means no information is lost at window boundaries, and it smooths the model's scoring over time.

2. Tech Stack — What Each Library Does

Library	Role in this pipeline
sounddevice	Opens a live microphone stream and calls your code back with new audio as it arrives (the 'streaming' component of the pipeline)
librosa	Loads audio files of any supported format and resamples/downmixes them to 16kHz mono in one call. Also does a lot of other audio processing stuff.
webrtcvad	Google's WebRTC voice activity detector. Classifies small (20ms) frames of audio as speech or non-speech.
soundfile	Reads/writes WAV/FLAC audio; used here to generate test audio and works alongside librosa.
numpy	All audio is represented as numpy float32 arrays — this is the common currency between every module in the pipeline.
FFmpeg (via Docker)	Handles decoding of compressed formats (MP3, M4A, OGG) that soundfile can't read natively; librosa/ffmpeg-python can't do this either.
pytest	Runs the automated test suite (tests/test_pipeline.py) to verify correctness.

On the datasets and papers you shared

ASVspoof 2019, MAVOS-DD, SEA-Spoof, the DEEP-VOICE links, Svarah, and the wav2vec2-XLS-R model are all relevant to **feature extraction and model training** — they contain labeled real/fake audio used to train the XGBoost classifier, or (in wav2vec2's case) could be used as a learned feature extractor instead of hand-crafted MFCC features. None of that is used in Sprint 1A: your job produces clean, windowed *audio*, not features or predictions. Whoever owns Sprint 1B will consume your output and reach for those resources next.

3. Project Structure

voice-ingestion/	
■■■ requirements.txt	Python dependencies
■■■ .gitignore	
■■■ README.md	Setup + run instructions
■■■ docker/	
■ ■■■ Dockerfile	Containerized environment (FFmpeg + Python deps)
■■■ src/	
■ ■■■ config.py	Shared constants – single source of truth
■ ■■■ file_loader.py	File input: any format -> 16kHz mono array
■ ■■■ mic_capture.py	Live microphone streaming
■ ■■■ vad.py	Voice activity detection (WebRTC)
■ ■■■ window_accumulator.py	Buffers speech into overlapping 1s windows
■ ■■■ pipeline.py	Wires it all together, entry point
■■■ tests/	
■ ■■■ test_pipeline.py	Automated correctness tests (pytest)
■■■ data/	Local test audio (gitignored)

4. config.py — The Shared Settings File

Every other file imports its constants from here. This is intentional: if 'window size' or 'sample rate' were hardcoded separately in five files, changing one would silently break the others. One file, one source of truth.

Key values

- **SAMPLE_RATE = 16000** — 16kHz is the team-agreed standard. It's high enough to capture speech frequencies clearly, but much lighter to process than 44.1kHz (CD quality), which matters for real-time performance.
- **WINDOW_DURATION_SEC = 1.0, WINDOW_OVERLAP = 0.5** — the sprint-plan requirements, translated into sample counts: *WINDOW_SIZE_SAMPLES = 16000* (1 second of audio at 16kHz) and *WINDOW_STRIDE_SAMPLES = 8000* (how far the window slides forward each time — half the window size, giving 50% overlap).
- **VAD_FRAME_MS = 20** — WebRTC VAD only accepts frames of exactly 10, 20, or 30ms. 20ms is the sprint-plan's choice, giving *VAD_FRAME_SAMPLES = 320* samples per frame.
- **VAD_AGGRESSIVENESS = 2** — WebRTC VAD's sensitivity dial, from 0 (lets more audio through as 'speech') to 3 (filters hardest). 2 is a reasonable middle ground to start with; it's the first thing to tune if you see too much silence leaking through, or too much real speech getting cut.

5. file_loader.py — Loading Pre-Recorded Audio

Handles the 'someone uploads a call recording' path. One function does all the work because librosa already knows how to decode every required format internally.

load_audio_file(filepath)

```
audio, _ = librosa.load(filepath, sr=SAMPLE_RATE, mono=(CHANNELS == 1))
```

This single line does three things at once:

- **Decodes** the file regardless of format (WAV/MP3/M4A/OGG/FLAC) — for compressed formats, librosa quietly delegates to FFmpeg/audioread under the hood.
- **Resamples** to 16kHz (`sr=SAMPLE_RATE`) — if the source file was recorded at, say, 44.1kHz, librosa mathematically resamples it down.
- **Downmixes** to mono (`mono=True`) — if the file is stereo, the two channels get averaged into one.

It also validates the extension against `SUPPORTED_FORMATS` and raises clear errors (*ValueError*, *FileNotFoundError*) for bad input, rather than failing with a confusing library-internal traceback.

get_audio_duration(audio)

A one-line helper: `len(audio) / SAMPLE_RATE`. Useful for logging/debugging ('this file loaded as 12.4 seconds') without repeating that math everywhere.

6. mic_capture.py — Live Microphone Streaming

This handles the 'live call' path — audio that doesn't exist as a file, but arrives continuously in real time. It uses what the sprint plan calls a **streaming callback pattern**: instead of recording a fixed clip and waiting for it to finish, you open a persistent stream, and the audio library calls your function automatically every time a new chunk is ready.

class MicrophoneStreamer

- **_callback(indata, frames, time_info, status)** — sounddevice invokes this automatically on a background audio thread whenever a new block of samples is captured. It just pushes the audio into a queue — kept minimal because callbacks need to return fast, or you'll drop audio.
- **start() / stop()** — open and close the underlying `sd.InputStream`.
- **chunks()** — a generator: your main code calls *for chunk in streamer.chunks()*: and gets each new piece of audio as it becomes available, pulled off the queue that the callback is filling in the background.

Why a queue between the callback and your code? The callback runs on a separate, high-priority audio thread that must never block. The queue lets your main program consume audio at its own pace without ever slowing down or interrupting the capture thread.

Note: this module needs an actual microphone device and cannot be exercised in a sandboxed/headless environment — test it on your own machine.

7. vad.py — Voice Activity Detection

Wraps Google's WebRTC VAD engine, which has a strict input requirement: 16-bit PCM audio, in frames of exactly 10/20/30ms. Our pipeline audio is float32 in the range [-1, 1] (the normal representation for processing), so there's a conversion step before every VAD call.

`_float_to_pcm16(frame)`

```
pcm16 = (frame * 32768.0).clip(-32768, 32767).astype(np.int16)
```

Float audio uses -1.0 to 1.0 to represent amplitude; 16-bit PCM uses integers from -32768 to 32767.

Multiplying by 32768 converts one range to the other; `.clip()` guards against rare rounding overshoot at the boundary.

`is_speech(frame)`

Validates the frame is exactly `VAD_FRAME_SAMPLES` (320 samples = 20ms) long, converts it to PCM16 bytes, and asks `webrtcvad`'s engine for a True/False speech classification.

`filter_speech(audio)`

The main entry point other modules use. Splits a full audio array into consecutive 20ms frames, classifies each one, and concatenates only the frames marked as speech — silence and non-speech noise are dropped entirely. Any leftover partial frame at the end (shorter than 20ms) is discarded, since VAD can't classify a fractional frame.

The built-in test at the bottom of the file proves this works correctly: pure silence returns 0 samples kept, while a non-silent synthetic tone gets partially classified as speech-like — verified when you ran `python3 vad.py` above.

8. window_accumulator.py — Overlapping Windows

The final stage: takes a continuous stream of VAD-filtered speech and slices it into fixed-size, overlapping windows — the format Sprint 1B's feature extraction expects.

`class WindowAccumulator`

Maintains an internal buffer (`self._buffer`) that grows as new audio is pushed in.

`push(audio_chunk)`

```
self._buffer = np.concatenate([self._buffer, audio_chunk])
while len(self._buffer) >= self.window_size:
    window = self._buffer[: self.window_size]
    windows.append(window.copy())
    self._buffer = self._buffer[self.stride:] # <- advance by STRIDE, not window_size
```

The key design detail is on the last line: after emitting a window, the buffer advances by **stride** (8000 samples / 0.5s) rather than by the full window size (16000 samples). That's what creates the 50% overlap — the next window reuses the second half of the previous one. If it advanced by the full window size instead, there would be no overlap at all.

This design also means it works identically whether you feed it all your audio at once, or in small streaming chunks (e.g. 100ms pieces from the microphone) — which is exactly what *test_window_accumulator_streaming_chunks* in the test suite verifies.

reset()

Clears the buffer — useful when a call ends or after a long silence, so old audio doesn't bleed into a new conversation's first window.

9. pipeline.py — Wiring It All Together

This is the file that actually runs (it's Docker's CMD). It connects the pieces above into the two real-world entry points: a file, or a live microphone.

process_file(filepath)

```
audio = load_audio_file(filepath)                # file_loader
speech_only = VoiceActivityDetector().filter_speech(audio)  # vad
windows = WindowAccumulator().push(speech_only)           # window_accumulator
return windows
```

Three lines, three modules, one clean output: a list of 1-second windows ready for feature extraction.

process_mic_stream(duration_sec)

The live equivalent: starts a MicrophoneStreamer, and for each incoming audio chunk, checks whether it's speech, pushes it into the accumulator, and *yields* any windows that become ready — a generator, so the caller can process windows as they arrive in real time rather than waiting for the whole stream to finish.

Like mic_capture.py, this function needs a real microphone and won't run in a sandboxed environment — verify it on your own machine.

10. tests/test_pipeline.py — What's Verified

Test	What it proves
test_config_values	Sample rate / window / stride constants match the sprint spec (16kHz, 1s window, 0.5s stride)
test_vad_rejects_silence	Pure silence produces zero speech samples — VAD is actually filtering.
test_vad_frame_length_validation	Passing a wrong-sized frame raises a clear error instead of failing silently.
test_window_accumulator_overlap_math	2.5s of audio correctly produces 4 overlapping windows of exactly 16000 samples each
test_window_accumulator_streaming_chunks	Pushing audio all at once vs. in small streaming chunks produces identical windows —
test_window_accumulator_reset	reset() actually empties the internal buffer.

All 6 tests passed when run in this session (`pytest tests/ -v`). Run them again on your own machine after setup to confirm your environment matches.

11. docker/Dockerfile — Containerized Environment

Handles the 'configure Docker to handle audio format conversions with FFmpeg' requirement from the sprint plan. A container guarantees your teammates (and the judges' demo machine) get the exact same environment you developed in — no 'works on my machine' problems.

- **FROM python:3.11-slim** — a lightweight Python base image.
- **apt-get install ffmpeg portaudio19-dev libsndfile1** — the system-level (non-Python) libraries these packages need: FFmpeg for decoding compressed audio formats, PortAudio for microphone access, libsndfile for WAV/FLAC I/O.
- **pip install -r requirements.txt** — installs the Python packages.
- **CMD ["python3", "src/pipeline.py"]** — what runs by default when the container starts.

Build and run with:

```
docker build -t voice-ingestion -f docker/Dockerfile .
docker run voice-ingestion
```

12. Running It On Your Own Machine

```
# 1. Create and activate a virtual environment
python3 -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate

# 2. Install system dependency (PortAudio) - required by sounddevice
brew install portaudio           # macOS
sudo apt-get install portaudio19-dev libsndfile1 ffmpeg    # Ubuntu/Debian

# 3. Install Python dependencies
pip install -r requirements.txt

# 4. Run the pipeline on a test file
cd src
python3 pipeline.py ../data/test_tone.wav

# 5. Run the test suite
cd ..
pip install pytest
pytest tests/ -v
```

13. Output Contract — What You Hand Off

This is the single most important thing to agree on with the rest of the team before everyone starts building in parallel:

```
Each window = np.ndarray
  shape: (16000,)
  dtype: float32
  values: [-1.0, 1.0]
  represents: exactly 1 second of 16kHz mono speech audio
```

Sprint 1B's feature extraction and Sprint 3's streaming/aggregation both consume exactly this shape. As long as your output matches this contract, they can build against fake/mock windows now and swap in

your real pipeline later without any integration surprises.